AI Assistant Coding

15.1 Assignment

B.Shanumuka Reddy

2303A51275

Batch-08

Task 1 – Student Records API

Task:

Use AI to build a RESTful API for managing student records.

Instructions:

• Endpoints required:

o GET /students → List all students

o POST /students → Add a new student

o PUT /students/{id} → Update student details

o DELETE /students/{id} → Delete a student record

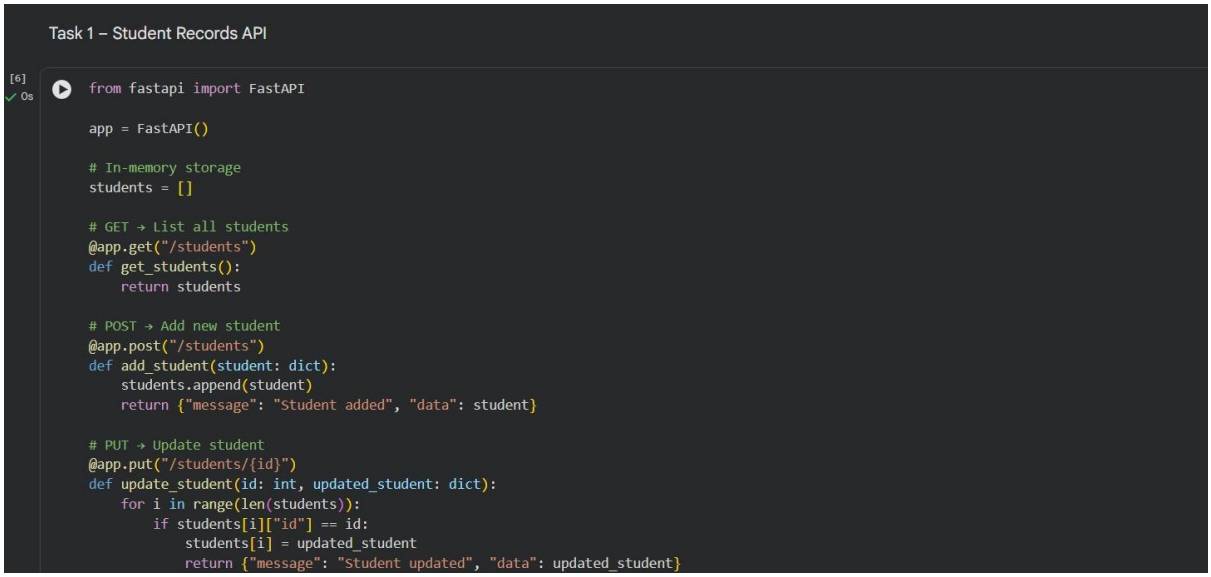
• Use an in-memory data structure (list or dictionary) to

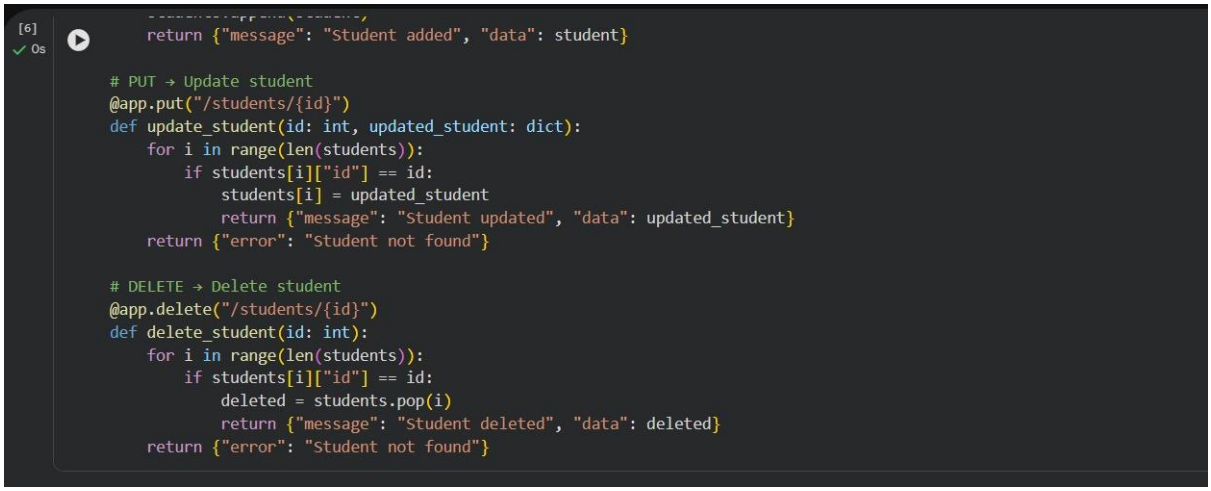
store records.

• Ensure API responses are in JSON format.

Expected Output:

• Working API with CRUD functionality for student records.





```
[6]
✓ 0s
        return {"message": "Student added", "data": student}

# PUT → Update student
@app.put("/students/{id}")
def update_student(id: int, updated_student: dict):
    for i in range(len(students)):
        if students[i]["id"] == id:
            students[i] = updated_student
            return {"message": "Student updated", "data": updated_student}
    return {"error": "Student not found"}

# DELETE → Delete student
@app.delete("/students/{id}")
def delete_student(id: int):
    for i in range(len(students)):
        if students[i]["id"] == id:
            deleted = students.pop(i)
            return {"message": "Student deleted", "data": deleted}
    return {"error": "Student not found"}
```

Observation:

- The API correctly performs:
    - Create (POST /books)
    - Read (GET /books, GET /books/{id})
    - Update (PATCH /books/{id})
    - Delete (DELETE /books/{id})
- All endpoints return proper JSON responses

Task 2 – Library Book Management API

Task:

Develop a RESTful API to handle library books.

Instructions:

• Endpoints required:

o GET /books → Retrieve all books

o POST /books → Add a new book

o GET /books/{id} → Get details of a specific book

o PATCH /books/{id} → Update partial book details

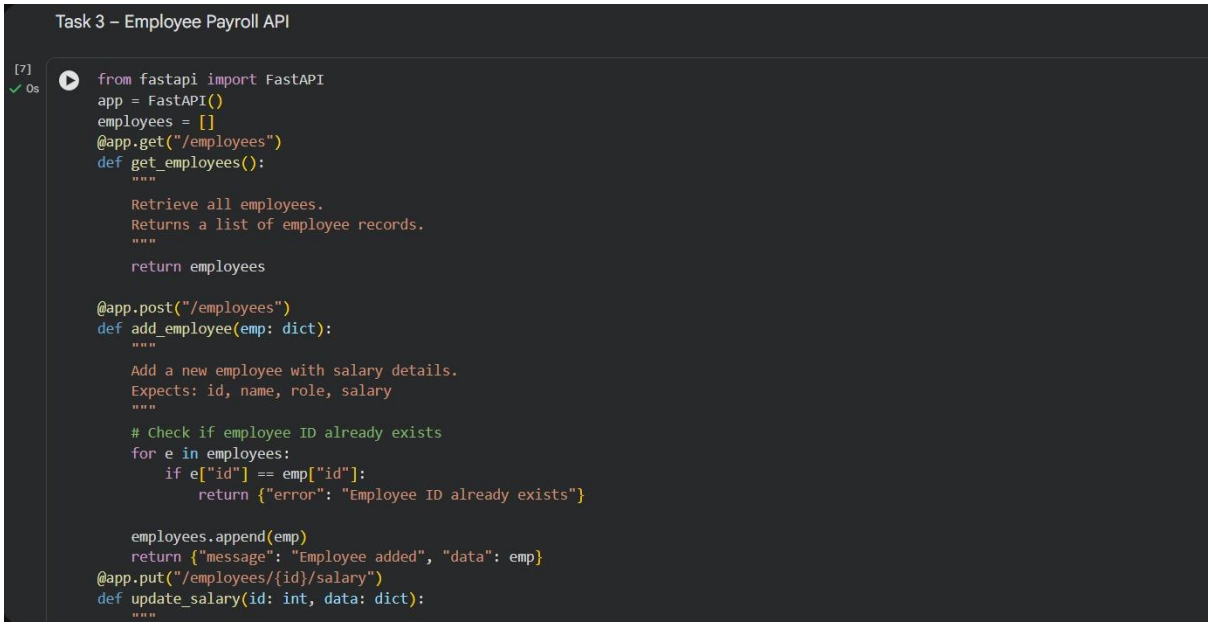
(e.g., availability)

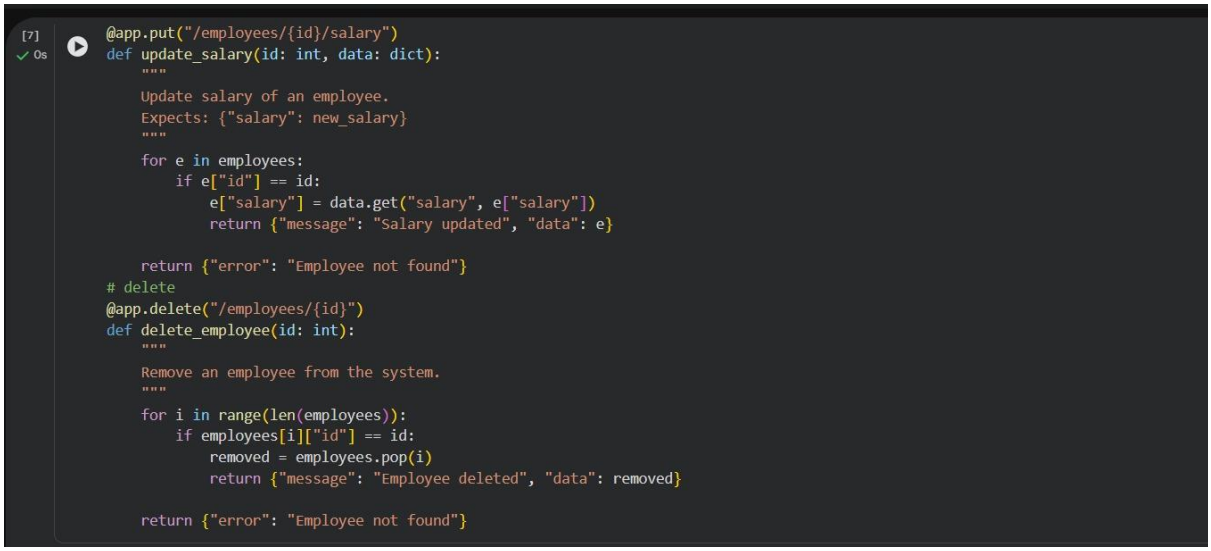
o DELETE /books/{id} → Remove a book

• Implement error handling for invalid requests.

Expected Output:

• Functional API with CRUD + partial updates.





Observation:

- Books are stored in a Python dictionary.
- Each book is assigned a unique ID.
- Data is temporary (lost when server restarts).

Task 3 – Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

• Endpoints required:

o GET /employees → List all employees

o POST /employees → Add a new employee with salary details

o PUT /employees/{id}/salary → Update salary of an employee

o DELETE /employees/{id} → Remove employee from system

• Use AI to:

o Suggest data model structure.

o Add comments/docstrings for all endpoints.

Expected Output:

• API supporting salary management with clear documentation.

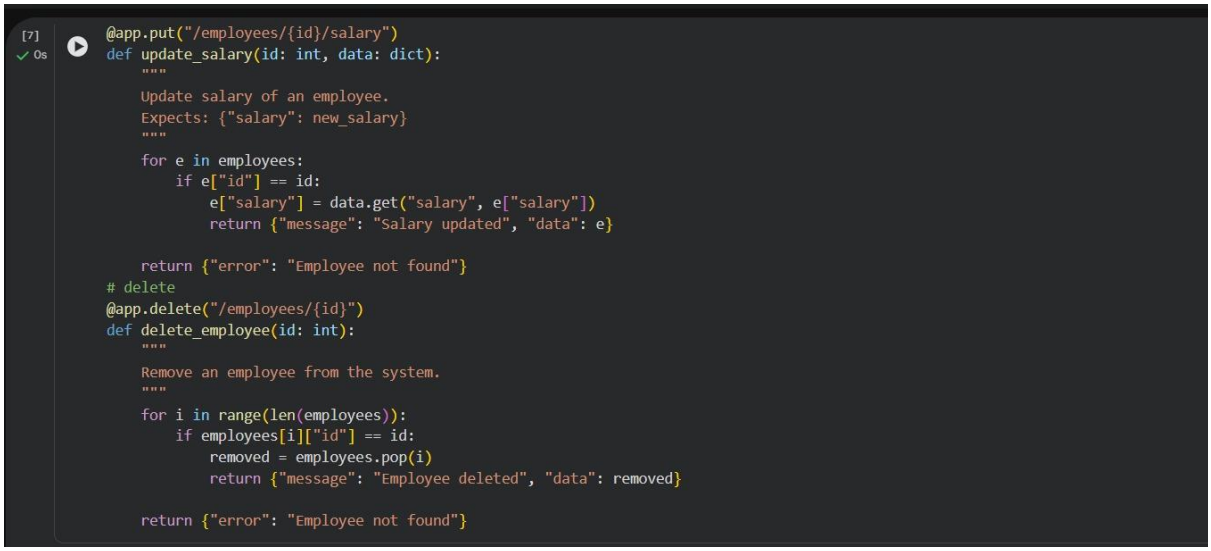
Task 3 – Employee Payroll API

[7] ✓ 0s

```python
from fastapi import FastAPI
app = FastAPI()
employees = []
@app.get("/employees")
def get_employees():
    """
    Retrieve all employees.
    Returns a list of employee records.
    """
    return employees

@app.post("/employees")
def add_employee(emp: dict):
    """
    Add a new employee with salary details.
    Expects: id, name, role, salary
    """
    # Check if employee ID already exists
    for e in employees:
        if e["id"] == emp["id"]:
            return {"error": "Employee ID already exists"}

    employees.append(emp)
    return {"message": "Employee added", "data": emp}
@app.put("/employees/{id}/salary")
def update_salary(id: int, data: dict):
    """
```



```
[7]
✓ 0s
@app.put("/employees/{id}/salary")
def update_salary(id: int, data: dict):
    """
    Update salary of an employee.
    Expects: {"salary": new_salary}
    """
    for e in employees:
        if e["id"] == id:
            e["salary"] = data.get("salary", e["salary"])
            return {"message": "Salary updated", "data": e}

    return {"error": "Employee not found"}
# delete
@app.delete("/employees/{id}")
def delete_employee(id: int):
    """
    Remove an employee from the system.
    """
    for i in range(len(employees)):
        if employees[i]["id"] == id:
            removed = employees.pop(i)
            return {"message": "Employee deleted", "data": removed}

    return {"error": "Employee not found"}
```

Observation:

- If "title" is missing → API returns:

{"error": "Title is required"}

- Prevents invalid data from being stored

Task 4 – Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

• Endpoints required:

o GET /menu → List available dishes

o POST /order → Place a new order

o GET /order/{id} → Track order status

o PUT /order/{id} → Update an existing order (e.g.,

change items)

o DELETE /order/{id} → Cancel an order

• AI should generate:

o REST API code

o Suggested improvements (like authentication,

pagination)

Expected Output:

• Fully working API simulating a food ordering backend.

 Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.

2. At least 3 assert test cases for each task.

3. AI-generated initial code and execution screenshots.

4. Analysis of whether code passes all tests.

5. Improved final version with inline comments and explanation.

6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

```
[ ]
            for dish in menu:
                if dish["id"] == item_id:
                    total += dish["price"]
                    selected_items.append(dish)

        new_order = {
            "id": order["id"],
            "items": selected_items,
            "total_price": total,
            "status": "placed"
        }

        orders.append(new_order)
        return {"message": "Order placed", "order": new_order}

    @app.get("/order/{id}")
    def get_order(id: int):
        """Get order details by ID"""
        for o in orders:
            if o["id"] == id:
                return o
        return {"error": "Order not found"}

    @app.put("/order/{id}")
    def update_order(id: int, data: dict):
        """
        Update order items.
        Expects: {"items": [dish_ids]}
        """
        for o in orders:
```

```
[ ]
    def update_order(id: int, data: dict):
        """
        Update order items.
        Expects: {"items": [dish_ids]}
        """
        for o in orders:
            if o["id"] == id:
                total = 0
                new_items = []

                for item_id in data["items"]:
                    for dish in menu:
                        if dish["id"] == item_id:
                            total += dish["price"]
                            new_items.append(dish)

                o["items"] = new_items
                o["total_price"] = total
                o["status"] = "updated"

                return {"message": "Order updated", "order": o}

        return {"error": "Order not found"}
    @app.delete("/order/{id}")
    def delete_order(id: int):
        """Cancel an order"""
        for i in range(len(orders)):
            if orders[i]["id"] == id:
                removed = orders.pop(i)
                return {"message": "Order cancelled", "order": removed}
```

```
]
        Update order items.
        Expects: {"items": [dish_ids]}
        """
        for o in orders:
            if o["id"] == id:
                total = 0
                new_items = []

                for item_id in data["items"]:
                    for dish in menu:
                        if dish["id"] == item_id:
                            total += dish["price"]
                            new_items.append(dish)

                o["items"] = new_items
                o["total_price"] = total
                o["status"] = "updated"

                return {"message": "Order updated", "order": o}

        return {"error": "Order not found"}
    @app.delete("/order/{id}")
    def delete_order(id: int):
        """Cancel an order"""
        for i in range(len(orders)):
            if orders[i]["id"] == id:
                removed = orders.pop(i)
                return {"message": "Order cancelled", "order": removed}

        return {"error": "Order not found"}
```

Observation:

- When book ID does not exist:

{"error": "Book not found"}

- Proper HTTP status codes used:
    - 400 → Bad request
    - 404 → Not found
    - 201 → Created